Deloitte.

In association with Cloutthat

GETTING STARTED GUIDE

EventZen

End-to-End Event Management Platform

📋 **NOTE**

This guide is intended for Deloitte and Cloutthat evaluators.

It walks through every step required to clone, configure, and run the full EventZen platform — from zero to a working application — using Docker Compose.

No prior knowledge of the codebase is required.

Repository	https://github.com/Snaju003/EventZen-Capstone-Project-Deloitte
Recommended Setup	Docker Compose (20+ containers — single command launch)
Estimated Setup Time	~10–15 minutes (first pull may take longer due to image downloads)
Frontend URL	http://localhost:5173
API Docs (Swagger)	http://localhost:3000/api-docs

SECTION 1: PREREQUISITES

Before starting, ensure the following software is installed on your machine. Docker Desktop is the only hard requirement for running EventZen — all other tools are only needed for local development outside of Docker.

Required Software

Tool	Min. Version	Purpose	Download
Docker Desktop	4.x+	Runs all 20+ containers	docker.com/products/docker-desktop
Docker Compose	v2+ (bundled)	Orchestrates the full stack	Included with Docker Desktop
Git	2.x+	Clone the repository	git-scm.com

Optional — Local Development Tools

Tool	Version	Used For
Node.js	20.x LTS	Running frontend or Node.js services locally
Java JDK	17	Running Event Service / Budget Service locally
Apache Maven	3.9+	Building Java services locally
.NET SDK	10.0	Running Booking Service locally

⚠ **IMPORTANT**

Docker Desktop must be running before you execute any docker compose commands.

On Windows, ensure WSL 2 backend is enabled in Docker Desktop settings for best performance.

On macOS with Apple Silicon (M1/M2/M3), Docker Desktop supports ARM natively — no extra configuration needed.

System Requirements

Resource	Minimum	Recommended
RAM	8 GB	16 GB (20+ containers running simultaneously)
Free Disk Space	10 GB	20 GB (Docker images + build cache)
CPU Cores	4 cores	8 cores for faster builds
OS	Windows 10, macOS 11, Ubuntu 20.04	Any modern 64-bit OS with Docker support

SECTION 2: REQUIRED CREDENTIALS & API KEYS

EventZen integrates with several external services. You will need active accounts and API keys for the services listed below before the platform can fully operate. The evaluator should have received these credentials separately.

> 📋 **NOTE**
> All secrets are injected into the platform via HashiCorp Vault — you do NOT place API keys directly into service config files.
> The seed script (Step 3) handles writing secrets into Vault automatically once you provide them in the .env file.

Service	Required Credentials	Purpose	Free Tier?
MongoDB Atlas	Connection URI (mongodb+srv://...)	Primary database for all services	Yes
Gmail / SMTP	Email address + App Password	OTP emails, booking confirmations	Yes (Gmail)
Cloudinary	Cloud Name + API Key + API Secret	Image uploads & CDN	Yes
Groq AI	API Key	AI event/venue description generation	Yes (free)

How to Obtain Each Credential

MongoDB Atlas

- Go to: cloud.mongodb.com
- Create a free M0 cluster → Database Access → Create a database user
- Network Access → Allow access from anywhere (0.0.0.0/0) for local testing
- Connect → Drivers → Copy the connection string (replace <password> with your DB user password)

Gmail App Password (SMTP)

- Enable 2-Factor Authentication on your Google account
- Go to: myaccount.google.com/apppasswords
- Create a new App Password for "Mail" → copy the 16-character password
- Use your full Gmail address as SMTP_USER and SMTP_FROM

Cloudinary

- Go to: cloudinary.com
- Register for a free account → Dashboard → copy Cloud Name, API Key, and API Secret

Groq AI

- Go to: console.groq.com
- Sign up → API Keys → Create a new API key → copy it

SECTION 3: INSTALLATION & SETUP

Follow these steps in order. Each step must be completed before moving to the next.

1 Clone the Repository

Open a terminal (Command Prompt, PowerShell, or Bash) and run:

```
git clone https://github.com/Snaju003/EventZen-Capstone-Project-Deloitte.git
cd EventZen-Capstone-Project-Deloitte
```

> ✔ **TIP**
> This clones the full monorepo including frontend, all backend services, Vault configuration, Docker Compose files, and monitoring stack.

2 Create the .env File

In the project root directory, create a file named .env (no extension). Copy the template below and fill in your credentials from Section 2:

```
# ─── MongoDB Atlas ──────────────────────────────────────────
MONGO_ATLAS_URI=mongodb+srv://<user>:<password>@<cluster>.mongodb.net/?appName=Clus
ter
MONGO_DATABASE=eventzen

# ─── JWT Secrets (use any long random strings) ──────────────
JWT_SECRET=your_super_secret_jwt_key_here_min_32_chars
JWT_REFRESH_SECRET=your_refresh_token_secret_here_min_32_chars
JWT_EXPIRES_IN=1h
JWT_REFRESH_EXPIRES_IN=1d

PAYMENT_CURRENCY=INR
CONVENIENCE_FEE_PERCENT=5

# ─── SMTP / Gmail ───────────────────────────────────────────
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your.email@gmail.com
SMTP_PASS=your_16char_app_password
SMTP_FROM=your.email@gmail.com

# ─── Cloudinary ─────────────────────────────────────────────
```

```
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret

# ─── Groq AI ──────────────────────────────────────────────
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxx
GROQ_MODEL=llama-3.3-70b-versatile

# ─── CORS / Frontend ──────────────────────────────────────
FRONTEND_URL=http://localhost:5173
CORS_ORIGINS=http://localhost:5173

# ─── Internal Service Auth ────────────────────────────────
INTERNAL_SERVICE_SECRET=any_random_secret_min_32_chars
INTERNAL_SIGNATURE_TTL_MS=60000

# ─── Rate Limiting ────────────────────────────────────────
AUTH_RATE_LIMIT_WINDOW_MS=900000
AUTH_RATE_LIMIT_MAX_ATTEMPTS=10
AUTH_OTP_RATE_LIMIT_MAX_ATTEMPTS=5
```

📋 **NOTE**

JWT_SECRET and JWT_REFRESH_SECRET can be any long random string (minimum 32 characters).

INTERNAL_SERVICE_SECRET is used for service-to-service HMAC authentication.

Generate secure random strings with: node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

3 Seed Secrets into HashiCorp Vault

EventZen uses HashiCorp Vault for secrets management. Before starting Docker Compose, you must seed the secrets from your .env file into Vault using the provided helper script.

On Linux / macOS

Make the script executable and run it:

```
chmod +x vault/scripts/seed-required-secrets.sh
./vault/scripts/seed-required-secrets.sh
```

On Windows (PowerShell)

```
.\vault\scripts\demo-seed-start-verify.ps1
```

⚠ **IMPORTANT**

This script starts a temporary Vault container, writes all secrets to the KV v2 engine at secret/eventzen/<service-name>, and then stops the temporary container.

You must run this BEFORE docker compose up, otherwise services will fail to start as they cannot retrieve their configuration.

If Vault already has secrets from a previous run, the script will overwrite them safely.

Should the initial script fail to run, first execute "docker compose up –build -d". Although it will return an error, this is expected. Afterward, proceed by running the HashiCorp Vault Script.

When running "docker compose up --build -d", it will pull the HashiCorp Vault from a Docker image. The terminal might need a restart afterward; otherwise, the HashiCorp Vault script will just open the ".\vault\scripts\demo-seed-start-verify.ps1" file.

4 Start the Platform with Docker Compose

EventZen includes two Docker Compose files for different use cases. Choose the one that matches your scenario:

	docker-compose.yml (Development)	docker-compose_prod.yml (Production)
Purpose	Local development & testing — builds images from source code	Pulls pre-built images from Docker Hub (snaju/*)
Image source	build: ./backend/<service> — builds locally	image: snaju/eventzen-<service>:latest — pulls from Docker Hub
When to use	When you have the source code and want to make changes	Quick evaluation / demo — no source code needed
First launch time	Slower — compiles Java, .NET, and Node.js services	Faster — images are already built and hosted on Docker Hub

Option A — Development (Build from Source)

Use docker-compose.yml when you have the source code cloned and want to build all images locally:

Standard Launch (Core Services Only)

```
docker compose up -d
```

Launch with Monitoring Stack (Prometheus + Grafana)

```
docker compose --profile monitoring up -d
```

Option B — Production / Evaluator (Pull from Docker Hub)

Use docker-compose_prod.yml when you want to run the platform without building from source. This pulls pre-built images from Docker Hub (snaju/*):

Standard Launch (Core Services Only)

```
docker compose -f docker-compose_prod.yml up -d
```

Launch with Monitoring Stack (Prometheus + Grafana)

```
docker compose -f docker-compose_prod.yml --profile monitoring up -d
```

📋 **NOTE**

The first launch may take 5–10 minutes as Docker pulls all base images (~2–3 GB total).

Subsequent launches are much faster as images are cached locally.

Services start in dependency order — Vault and Kafka initialise first, followed by backend services, then the frontend.

The prod compose file uses pre-built images: snaju/eventzen-auth-service:latest, snaju/eventzen-event-service:latest, etc.

5 Verify All Services Are Running

After docker compose up completes, verify all containers are healthy:

```
docker compose ps
```

All containers should show a Status of running or healthy. If any container shows exited or restarting, check its logs:

```
docker compose logs -f <service-name>

# Examples:
docker compose logs -f eventzen-auth-service
docker compose logs -f eventzen-event-service
docker compose logs -f eventzen-api-gateway
```

📋 **NOTE**

Spring Boot services (Event Service, Budget Service) take ~30–60 seconds to fully start.

.NET Booking Service takes ~20–30 seconds.

Node.js services start almost immediately.
Wait for all services to show healthy before navigating to the frontend.

SECTION 4: ACCESSING THE APPLICATION

Once all containers are running and healthy, the full EventZen platform is accessible at the following URLs:

Service	URL	Notes
Frontend (React App)	http://localhost:5173	Main application — start here
API Gateway	http://localhost:3000	All API calls pass through here
Swagger / API Docs	http://localhost:3000/api-docs	Interactive API documentation
HashiCorp Vault UI	http://localhost:8200	Secrets management (token: root)
Prometheus	http://localhost:9090	Metrics dashboard (monitoring profile)
Grafana	http://localhost:3100	Service dashboards (monitoring profile)
Kafka (external)	localhost:9094	Kafka broker for dev tools
Auth Service (direct)	http://localhost:3001	Auth microservice (via Gateway preferred)
Event Service (direct)	http://localhost:8080	Java/Spring Boot event service
Booking Service (direct)	http://localhost:5000	.NET booking service
Payment Service (direct)	http://localhost:3002	Node.js payment service
Budget Service (direct)	http://localhost:8081	Java/Spring Boot budget service
Notification Service (direct)	http://localhost:3003	Node.js notification service

Default Test Accounts

For evaluation purposes, you can register new accounts directly in the application. Use the roles below to explore each user type:

Role	How to Access	Capabilities
Customer	Register normally at /register with role: Customer	Browse events, book tickets, make payments, view bookings
Admin	Register with role: Admin (or promote via API)	Create events/venues, manage vendors, view revenue, manage all bookings
Vendor	Register → Request vendor role upgrade → Admin approves	Accept/decline event assignments, scan QR codes for check-in

> ⚠ **IMPORTANT**
> OTP verification is required during registration. Ensure your SMTP credentials are correctly configured in .env so emails can be sent.
> For testing without real email, you can temporarily check the MongoDB Atlas USERS collection to read the OTP directly from the database.

Exploring the Platform — Recommended Evaluation Flow

1. Customer Journey

1. Register a new Customer account → verify OTP from email
2. Browse Events on the public events page
3. View Event Details — see venue map, ticket tiers, and image carousel
4. Book a Ticket — select ticket type and proceed to checkout
5. View My Bookings — download PDF ticket or view QR code
6. Check Notifications — booking confirmation notification appears

2. Admin Journey

7. Login as Admin → access Admin Dashboard
8. Create a Venue — fill in details and generate AI description
9. Create an Event — assign venue, add ticket tiers, assign vendor
10. Publish the Event — lifecycle: Draft → Pending Vendor → Published
11. View Revenue — see convenience fee revenue per event
12. Manage Attendees — view bookings and check-in status per event
13. Approve Vendor Requests — promote users to vendor role

3. Vendor Journey

14. Request Vendor Role from profile page → Admin approves
15. Login as Vendor → view assigned events
16. Accept/Decline event assignment from the vendor dashboard
17. Check-In Attendees — use the QR scanner to scan customer QR tickets

SECTION 5: USEFUL DOCKER COMMANDS

The following commands are useful for managing and inspecting the running EventZen platform. When using the production compose file, append -f docker-compose_prod.yml to each command.

Viewing Logs

```
# Stream logs from a specific service
```

```
docker compose logs -f eventzen-api-gateway
docker compose logs -f eventzen-event-service
docker compose logs -f eventzen-booking-service

# View last 100 lines from all services
docker compose logs --tail=100

# View Vault logs
docker compose logs -f eventzen-vault
```

Checking Service Status

```
# Show all container statuses and ports
docker compose ps

# Check resource usage (CPU, memory)
docker stats
```

Restarting a Specific Service

```
# Restart just the API Gateway (e.g., after config change)
docker compose restart eventzen-api-gateway

# Rebuild and restart a single service (dev compose only)
docker compose up -d --build eventzen-auth-service
```

Stopping the Platform

```
# Stop all containers (preserves volumes / data)
docker compose down

# Stop all containers AND remove volumes (clean slate)
docker compose down -v

# Stop monitoring stack only
docker compose --profile monitoring down
```

Running Tests

```
# Frontend tests
cd frontend && npm test

# API Gateway tests
cd backend/api-gateway && npm test
```

```
# Auth Service tests
cd backend/auth-service && npm test

# Event Service tests (Java)
cd backend/event-service && mvn test

# Booking Service tests (.NET)
cd backend/booking-service/tests/BookingService.Tests && dotnet test
```

SECTION 6: TROUBLESHOOTING

Problem	Likely Cause	Solution
Container keeps restarting	Vault secrets not seeded	Run the seed script (Step 3) before docker compose up
Frontend shows blank page	API Gateway not yet healthy	Wait 60 seconds, then refresh. Check: docker compose logs eventzen-api-gateway
OTP email not received	Incorrect SMTP credentials	Verify SMTP_USER, SMTP_PASS, and SMTP_HOST in .env; re-run seed script
Payment fails at checkout	Invalid keys	Ensure test mode keys are used (rzp_test_...). Check payment service logs.
Event Service not starting	Java/Spring Boot slow boot	Wait 60 seconds. Spring Boot takes longer to initialise than Node.js services.
Port already in use	Another service on same port	Stop the conflicting service or change the port in the compose file.
docker compose not found	Old Docker version	Update to Docker Desktop 4.x+ which bundles Compose v2 as docker compose (no hyphen)
Vault shows access denied	Vault not initialised	Run the seed script again. Check: docker compose logs eventzen-vault
Images not uploading	Invalid Cloudinary credentials	Re-check CLOUDINARY_CLOUD_NAME, API_KEY, and API_SECRET in .env
AI description not generating	Invalid Groq API key	Verify GROQ_API_KEY is valid at console.groq.com. Feature gracefully degrades if unavailable.
Prod image pull fails	Not logged in to Docker Hub	Run: docker login -u snaju then retry docker compose -f docker-compose_prod.yml up -d

Common Log Locations

Service	Log Command
API Gateway	docker compose logs -f eventzen-api-gateway
Auth Service	docker compose logs -f eventzen-auth-service
Event Service (Java)	docker compose logs -f eventzen-event-service
Booking Service (.NET)	docker compose logs -f eventzen-booking-service

Payment Service	docker compose logs -f eventzen-payment-service
Budget Service (Java)	docker compose logs -f eventzen-budget-service
Notification Service	docker compose logs -f eventzen-notification-service
HashiCorp Vault	docker compose logs -f eventzen-vault
Apache Kafka	docker compose logs -f eventzen-kafka
Frontend (Nginx)	docker compose logs -f eventzen-frontend

SECTION 7: QUICK REFERENCE CARD

A concise summary of the most important commands and URLs for quick access during evaluation.

Start / Stop

Development (build from source — docker-compose.yml):

```
docker compose up -d                              # Start everything
docker compose --profile monitoring up -d         # Start with Grafana + Prometheus
docker compose down                               # Stop (keep data)
docker compose down -v                            # Stop + wipe all data
```

Production / Evaluator (pull from Docker Hub — docker-compose_prod.yml):

```
docker compose -f docker-compose_prod.yml up -d
docker compose -f docker-compose_prod.yml --profile monitoring up -d
docker compose -f docker-compose_prod.yml down
```

Check Health

```
docker compose ps                     # Container statuses
docker compose logs -f <service>      # Live logs for a service
docker stats                           # CPU / memory usage
```

Application URLs

URL	Service
http://localhost:5173	Main Frontend (React App) — start here
http://localhost:3000/api-docs	Swagger API Documentation
http://localhost:8200	HashiCorp Vault UI (token: root)
http://localhost:9090	Prometheus (monitoring profile)
http://localhost:3100	Grafana Dashboards (monitoring profile)